

Gurudatta R K

☎ +91-8424016635 | ✉ gurudatta.kondampallikar@gmail.com | 🐙 github:GurudattaRK | 🔗 LinkedIn

Systems programmer with hands-on experience in x86-64 assembly, hardware PMU instrumentation, and latency-critical system design; targeting low-latency developer and HPC roles.

TECHNICAL SKILLS

Languages: C, C++, x86-64 Assembly, CUDA C, Python

Low-Latency & Performance: SIMD/AVX Intrinsics, Branchless Programming, Loop Unrolling, Memory Alignment, Cache-Friendly Data Layouts, LTO/PGO, Branch Elimination, Pipeline Optimization

CPU Architecture: x86-64 Microarchitecture, PMU Hardware Counters, Instruction-Level Parallelism (ILP), Out-of-Order Execution, Cache Hierarchy (L1/L2/L3), TLB Internals, RDTSC/RDTSCP

Systems & OS: Linux `perf_event_open(2)`, `ioctl` counter control, position-independent shared libraries

Heterogeneous Parallel Computing: Multithreading, Multi-core Parallelism, GPU Parallelism (CUDA)

Profiling & Debugging Tools: PicoPerf (author – custom zero-overhead PMU benchmarking library), Linux `perf_event` API, GDB, Valgrind/Callgrind, `perf stat/record`

PROJECTS

•PicoPerf – Zero-Overhead PMU Benchmarking Library

🐙 [GitHub](#)

- Captures **hardware-native CPU metrics** directly from x86-64 CPU hardware registers – retired instructions, elapsed cycles, **IPC**, branch misses, **L1D/L1I cache misses**, and **DTLB/ITLB misses** – with no sampling, no interpolation, and **no loss of resolution** at the nanosecond and single-cycle level.
- Uses **RDTSC/RDTSCP** x86 instructions with **lfence** serialization for nanosecond-precision cycle measurement; answers not just *how long* a code path takes but *why* – independently isolating cache pressure, branch mispredictions, and IPC degradation as separate measurable hardware signals.
- Counters opened as a **perf group** (leader + 8 members) – all 9 start and stop atomically via a single group-enable `ioctl` on the leader, guaranteeing every counter covers the **exact same instruction window** with zero inter-counter skew; FDs kept live across iterations to eliminate per-call syscall cost.

•TeraCrypt – Multi-core x86-64 Assembly Encryption Engine

🐙 [GitHub](#)

- Implemented the cipher **entirely in x86-64 Assembly**, compiled into a **position-independent shared library** (.so); exposed to a Python frontend via a thin **ctypes wrapper** – assembly executes directly with zero Python interpreter overhead on the hot path.
- Python **detects physical core count** and distributes file chunks uniformly, with each core running cipher in assembly as an independent process with its own memory space for **lock free and zero synchronization overhead**; achieved **450–500% speedup** on a 6-core machine with throughput scaling **linearly with core count**, validated at 2, 4, and 6 cores.
- Applied **register-level control**, manual calling conventions, **loop unrolling**, and **branch elimination** at the assembly level to minimize pipeline stalls and saturate the CPU's integer execution units.

•CUDA Cipher – GPU-Parallelized Encryption Kernel

🐙 [GitHub](#)

- Designed CUDA kernel with **coalesced global memory access** and **dynamic thread-block / warp configuration** tuned at runtime to available GPU architecture; benchmarked against CPU (C and Assembly) baselines to quantify **throughput vs. latency trade-offs** between the GPU and CPU execution models.

•C Reference Cipher – Compiler-Optimized Encryption Library

🐙 [GitHub](#)

- Portable C reference implementation; applied **compile-time optimizations** (LTO, `-O3`, `-ffast-math`, `-march=native`) and structured code for **compiler-friendly generation** – inspected output assembly to verify optimal instruction sequences.

EXPERIENCE

•Software Developer at QUALYS

June 2025 – Present

- Identified and patched **multiple CVEs** in the macOS agent and its deployment scripts; hardened agent pipelines against **injection attacks** via input sanitization and systematic code hardening.
- Led the design and integration of **Software Composition Analysis (SCA)** into the macOS agent – implemented static and runtime dependency scanning, added a **CPU usage limiter** to prevent agent performance regressions, and built **custom macOS scanners** to extend detection coverage.

•Security Software Intern at SENTINEL ONE

June 2024 – January 2025

- Performed **reverse engineering** of newly discovered macOS malware samples and authored **4 new detection rules** covering previously undetected threat families, directly expanding the production detection engine's coverage.
- Optimized existing detection rule logic by refining filter conditions and improving detection accuracy; achieved up to **86% reduction in false positives** for legitimate macOS applications in the highest-frequency detection paths.

- Best Final Year Project 2024** – Awarded by PES University for post-quantum cryptography research project.
- Hackathon Events** – Winner of a Mathematics-based hackathon; Top 5 in 3 CTF hackathon events.

EDUCATION

• Bachelor of Technology in Computer Science Engineering <i>PES University, Bengaluru</i>	<i>November 2020 – December 2024</i>
---	--------------------------------------